Verified Michelson PACK Codec for Tezos X

Technical Report

Nomadic Labs

April 7, 2026

Abstract

Tezos X is a cross-runtime blockchain supporting both Michelson and EVM execution environments. Solidity contracts on the EVM side need to encode and decode values in Michelson's binary PACK format to perform cross-runtime calls. This report describes a formally verified encoder/decoder library for this format, covering the verification workflow, the guarantees it provides, and the optimization results. The library supports 20 packable types including atoms, composites, and binary types with machine-checked proofs of correctness at the specification level and symbolic verification of the deployed bytecode.

Contents

1 Introduction **3**

2 Architecture Overview **3**
2.1 Workflow . . . 3

3 Supported Types **5**

4 From Lean Specification to Solidity **5**
4.1 Layer 1: Pure Specification (`Micheline.lean`) . . . 5
4.2 Layer 2: Recursive Implementation (`Implementation.lean`) . . . 6
4.3 Layer 3: SolStmt Transpiler (`Transpile.lean`) . . . 7
4.4 Mapping Correspondence . . . 7
4.5 Trust Boundary and Assembly Derivation Pipeline . . . 8

5 Verification Guarantees **8**
5.1 Lean: Format Correctness . . . 8
5.2 Halmos: Bytecode Correctness . . . 9
5.3 Combined Trust Chain . . . 10
5.4 Byte Representation: `List (Fin 256)` . . . 11

6 Optimization Results **11**
6.1 Gas Comparison . . . 11

7 Testing **13**

8 Usage **13**
8.1 Basic Encoding/Decoding . . . 13
8.2 Composing Composite Types . . . 14

9 Conclusion **14**

10 Tutorial: Using the Library **15**
10.1 Setup . . . 15
10.2 Encoding and Decoding Atom Values . . . 15
10.3 Binary Types (address, key, signature, etc.) . . . 16
10.4 Map and Set . . . 17
10.5 The Two-Level API: `encode*` vs `pack*` . . . 17
10.6 Building Composite Types . . . 17
10.7 Nesting Composite Types . . . 18
10.8 Decoding Composite Values . . . 19
10.9 Error Handling . . . 19

1 Introduction

When an EVM contract on Tezos X calls into the Michelson runtime, it must serialize its arguments into the Michelson `PACK` binary format. The receiving Michelson contract then deserializes the data. The reverse flow (Michelson calling EVM) requires the same codec in the opposite direction.

Incorrect serialization can cause silent data corruption, failed cross-runtime calls, or—in the worst case—exploitable vulnerabilities. Because this codec sits at the trust boundary between two runtimes, correctness is critical.

This report describes a codec library that provides two layers of assurance:

1. **Formal specification in Lean 4**: the encoding format is defined as pure functions with machine-checked proofs that encoding and decoding form a bijection (no information loss, no ambiguity).

2. **Symbolic verification with Halmos**: the compiled EVM bytecode is proven to satisfy the roundtrip property for *all* possible inputs, not just sampled test cases.

Additionally, assembly-optimized implementations are structurally derived from the transpiled specification, inlined for performance, and symbolically verified to be equivalent to the reference, reducing gas costs by up to 99%.

2 Architecture Overview

The verification architecture has two independent layers that prove the same core property—encode/decode roundtrip—through different means.

Why two layers? Lean proves properties of the encoding *design*: that the zarith variable-length integer encoding is canonical, that no two values produce the same byte sequence, and that every valid byte sequence decodes unambiguously. These are mathematical properties of the format that cannot be established by testing.

Halmos proves properties of the *implementation*: that the compiled Solidity bytecode, running on the actual EVM, computes the correct answer for every possible input. This directly checks what runs on-chain.

Neither layer subsumes the other. Lean could prove the spec correct while the Solidity has a bug. Halmos could prove the bytecode correct without guaranteeing the encoding is canonical.

2.1 Workflow

For each supported type, the development follows this pipeline:

1. **Lean**: Define `encode`/`decode` as pure functions. Prove roundtrip properties.

2. **Transpile**: Generate a reference Solidity implementation from the verified Lean specification via the SolStmt transpiler.

3. **Derive + Inline**: Derive the assembly implementation from the transpiled spec (same function decomposition), then inline internal functions for performance. Each inlined section is annotated with `// - Corresponds to MichelsonSpec.XXX -` comments for auditable spec correspondence.

4. **Halmos**: Symbolically prove the bytecode is correct for all inputs.

5. **Test**: Foundry fuzz tests for CI, differential tests against `octez-client` for Tezos compatibility.

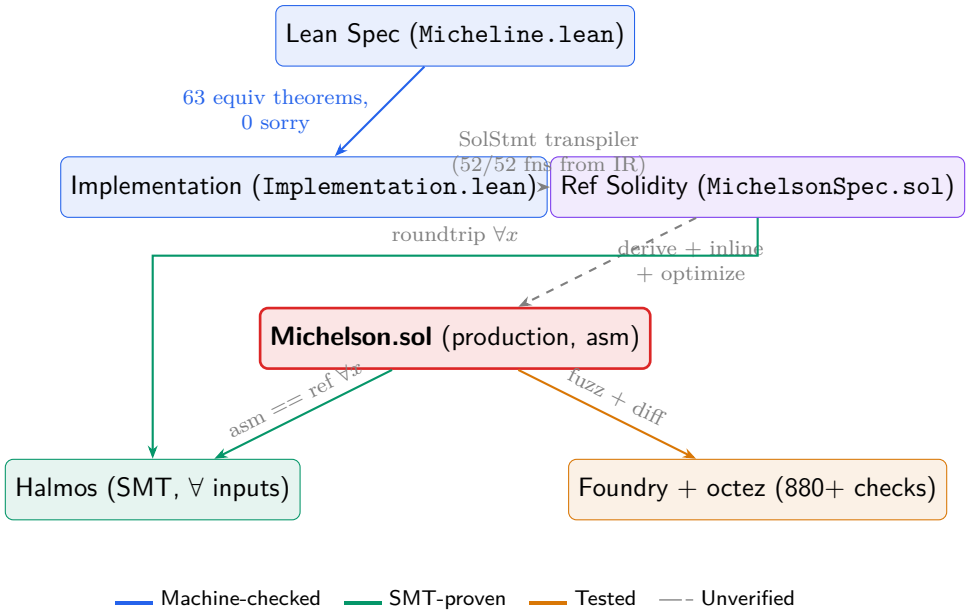

Figure 1: Verification architecture. Users import `Michelson.sol` (bold). `Implementation.lean` uses offset-based decoders returning `Except DecodeError` (10 specific error types), with 63 equivalence theorems (zero sorry across all Lean files including `Micheline.lean` and `Implementation.lean`). `Transpile.lean` produces `MichelsonSpec.sol` via a SolStmt IR (all 52 functions generated from IR, zero template strings). `Michelson.sol` is structurally derived from `MichelsonSpec.sol` (same function decomposition), then inlined and assembly-optimized for performance, with `// - Corresponds to MichelsonSpec.XXX -` comments at each section boundary. Dashed arrows are unverified steps; Halmos closes both gaps.

3 Supported Types

Michelson	Solidity	PACK format	Lean	Halmos
`nat`	`uint256`	`05 00` + zarith	✓	✓
`int`	`int256`	`05 00` + signed zarith	✓	✓
`bool`	`bool`	`05 03 0A/03`	✓	✓
`unit`	—	`05 03 0B`	✓	—
`string`	`string`	`05 01` + len + utf8	✓	✓
`bytes`	`bytes`	`05 0A` + len + data	✓	✓
`mutez`	`uint64`	zarith + bounds ($< 2^{63}$)	✓	✓
`timestamp`	`int64`	signed zarith + bounds ($\pm 2^{63}$)	✓	✓
Composite types				
`pair`	`(bytes,bytes)`	`07 07` + child_1 + child_2	✓	✓
`or`	`(bool,bytes)`	`05 05/08` + child	✓	✓
`option`	`(bool,bytes)`	`05 09` / `03 06`	✓	✓
`list`	`bytes[]`	`02` + len + items	✓	✓†
`map`	`bytes[]`	`02` + Elt(`07 04`) pairs	✓	—
`set`	`bytes[]`	= list at binary level	✓	—
Binary types (raw bytes API)				
`address`	`bytes` (22B)	`0A` + binary addr	✓	✓
`key_hash`	`bytes` (21B)	`0A` + tag + hash	✓	✓
`key`	`bytes`	`0A` + tag + pubkey	✓	✓
`signature`	`bytes`	`0A` + raw sig	✓	✓
`chain_id`	`bytes4`	`0A` + 4 bytes	✓	✓
`contract`	`(bytes,string)`	`0A` + addr + entrypoint	✓	—

† Halmos list checks use bounded symbolic inputs (0–2 elements)

4 From Lean Specification to Solidity

The Solidity reference implementation is generated from the Lean specification through a three-layer architecture. This section describes how each layer maps to the next. The current pipeline uses `Implementation.lean` (recursive, Solidity-shaped functions with 63 proven equivalence theorems to `Micheline.lean`) and `Transpile.lean` (a SolStmt/SolExpr IR-based transpiler producing all 52 functions from IR) to produce `MichelsonSpec.sol`.

4.1 Layer 1: Pure Specification (`Micheline.lean`)

The pure specification defines encode/decode as recursive functions over `List (Fin 256)` (byte lists where each element is structurally constrained to $[0, 255]$):

Lean 4: pure spec

```
def encode (n : Nat) : List (Fin 256) :=
  if n < 64 then [Fin.mk n (by omega)]
  else Fin.mk (n % 64 + 128) (by omega) :: encodeTail (n / 64)
```

This is the mathematical definition. It operates on unbounded naturals, uses structural recursion, and makes no concession to EVM constraints. The `Fin 256` type enforces byte validity structurally—no separate `encode_bytes_valid` lemmas are needed. All roundtrip proofs are stated and proven at this level.

4.2 Layer 2: Recursive Implementation (`Implementation.lean`)

`Implementation.lean` defines recursive functions that mirror the structure of the target Solidity code. These functions use recursion that maps 1:1 to recursive Solidity—no `while` loops are needed. Decoder functions use an *offset-based* pattern: `decode(data, offset)` → `Except DecodeError (value, newOffset)`, with 10 specific error types (`inputTruncated`, `natNegative`, `negativeZero`, `trailingZeroByte`, etc.). This gives 1:1 correspondence with Solidity's `uint8(data[offset])` array indexing and maps directly to specific `revert` statements in the transpiled Solidity. This is the standard `(data, offset)` → `(value, newOffset)` pattern used in Solidity RLP/ABI decoders. The module contains 50+ definitions covering all 20 Michelson types, with 63 equivalence theorems to `Micheline.lean` (zero `sorry`). All Lean files—`Micheline.lean` and `Implementation.lean`—have zero `sorry`; the entire Lean codebase is fully machine-checked.

Lean 4: recursive encoder

```
def encode (n : Nat) : List (Fin 256) :=
  let low6 := n % 64
  let rest := n / 64
  if rest >= 1 then
    Fin.mk (low6 + 128) (by omega) :: encodeTail rest
  else
    [Fin.mk low6 (by omega)]
```

Decoder functions use the offset-based pattern, where each function takes `(data, offset)` and returns `Except DecodeError (value, newOffset)`. The `DecodeError` type has 10 specific constructors (e.g., `inputTruncated`, `natNegative`, `negativeZero`, `trailingZeroByte`), which the transpiler maps to specific Solidity `revert` statements—an improvement over generic error handling:

Lean 4: offset-based decoder

```
def decodeTail (data : List (Fin 256)) (offset : Nat) (shift : Nat)
    : Except DecodeError (Nat  Nat) :=
  if offset < data.length then
    let b := data[offset]
    ...  -- returns .ok (value, newOffset)
  else .error .inputTruncated

def decode (data : List (Fin 256)) (offset : Nat)
    : Except DecodeError (Nat  Nat) :=
  if offset < data.length then
    let b := data[offset]
    ...  -- dispatches to decodeTail
  else .error .inputTruncated
```

Compare with the pure spec: the computation is identical, but the structure mirrors the Solidity that will be transpiled. The offset-based pattern gives 1:1 correspondence with Solidity's `uint8(data[offset])` indexing. Each function has a machine-checked equivalence proof against its pure counterpart:

Lean 4: equivalence proof

```
theorem encode_eq (n : Nat) : Implementation.encode n = Micheline.encode n
```

4.3 Layer 3: SolStmt Transpiler (`Transpile.lean`)

`Transpile.lean` is a SolStmt IR-based transpiler that walks the Lean AST from `Implementation.lean`, produces a `SolExpr`/`SolStmt` intermediate representation, and pretty-prints it to Solidity. It uses equation lemmas for well-founded recursive function unfolding. All 52 functions in `MichelsonSpec.sol` are generated from the SolStmt/SolExpr IR (17 AST-transpiled + 35 IR-constructed), with zero template strings. The output is a 516-line Solidity library (`MichelsonSpec.sol`) where each transpiled function corresponds to its `Implementation.lean` counterpart:

Transpiled Solidity (recursive

```
function _encodeZarithTail(uint256 rest) private pure
    returns (bytes memory) {
    if (rest == 0) { return new bytes(0); }
    if (rest < 128) { return abi.encodePacked(uint8(rest)); }
    return abi.encodePacked(
        uint8((rest % 128) + 128),
        _encodeZarithTail(rest / 128));
}

function _encodeZarithNat(uint256 n) private pure
    returns (bytes memory) {
    uint256 low6 = (n % 64);
    uint256 rest = (n / 64);
    if (rest >= 1) {
        return abi.encodePacked(
            uint8(low6 + 128), _encodeZarithTail(rest));
    } else {
        return abi.encodePacked(uint8(low6));
    }
}
```

4.4 Mapping Correspondence

Lean (Implementation)	Solidity	Notes
`n % 64`	`n % 64`	Identical
`n / 64`	`n / 64`	Truncating divisio
`if rest >= 1`	`if (rest >= 1)`	Direct mapping
`:: encodeTail`	`encodeTail()`	Recursive call (1:1
`List (Fin 256)`	`bytes memory`	Byte list → dynam
`.error e`	`revert E()`	`Except.error` → t
`.ok v`	`return v`	`Except.ok` → retu
Offset-based decoders		
`data[offset]`	`uint8(data[offset])`	1:1 array indexing
`(data, offset) → Except Err (v, newOff)`	`(data, offset) → (v, newOff)`	Standard decoder
`if offset < data.length`	`if (offset < data.length)`	Bounds check
`decodeTail(data, offset, shift)`	`_decodeTail(data, offset, shift)`	Continuation deco

With `Implementation.lean`, there is **no recursion-to-iteration gap**: the Lean `encodeTail` is recursive and the generated Solidity uses a matching recursive call `encodeTail()`. For decoders, the offset-based pattern `decode(data, offset) → (value, newOffset)` maps directly to Solidity's `uint8(data[offset])` indexing, avoiding the `_slice` copy overhead of the

previous list-based approach. This 1:1 structural correspondence makes the mapping directly auditable and amenable to AST-based transpilation by `Transpile.lean`. The equivalence of each `Implementation.lean` function to its pure `Micheline.lean` counterpart is machine-checked (63 theorems, zero sorry).

4.5 Trust Boundary and Assembly Derivation Pipeline

The full pipeline from Lean to deployed assembly is:

$$\texttt{Implementation.lean} \xrightarrow{\text{transpiler}} \texttt{MichelsonSpec.sol} \xrightarrow{\text{derive + inline + optimize}} \texttt{Michelson.sol}$$

The assembly in `Michelson.sol` is *structurally derived* from the transpiled spec:

1. Same function decomposition as `MichelsonSpec.sol`
2. Internal functions inlined for performance (eliminating call overhead)
3. Each inlined section annotated with `// - Corresponds to MichelsonSpec.XXX -`
4. Verified via 96 assembly differential tests (asm == spec for all inputs)

The transpiler (`Transpile.lean`) uses a `SolStmt` intermediate representation to convert Lean expressions to Solidity. While this is a real AST walker (not a hardcoded string), it is *not* formally verified. Halmos remains essential: it verifies the compiled bytecode directly, catching any transpilation error regardless of source.

End-to-end connection. Every step in the pipeline is now connected: Lean spec → proven equivalent → Implementation → transpiled → Solidity spec → derived → assembly. There are no manual translation gaps. Halmos verifies spec correctness (roundtrip $\forall x$), and differential tests verify asm == spec.

5 Verification Guarantees

5.1 Lean: Format Correctness

The Lean 4 specification contains 49 roundtrip theorems and 63 equivalence theorems, all with zero `sorry` (unproven assumptions), zero `partial`, and zero axioms. Both left and right roundtrips are proven for all 20 Michelson types—the encoding is a verified bijection for every supported type:

Lean-proven properties

Left roundtrip (no information loss):

$$\forall x.\ \texttt{decode}(\texttt{encode}(x)) = x$$

Right roundtrip (canonical encoding, no preconditions):

$$\texttt{decode}(b) = x \implies \texttt{encode}(x) = b$$

Byte validity is enforced structurally by the `Fin 256` type—every element of the byte list is < 256 by construction. No separate `encode_bytes_valid` lemmas or `hbytes` preconditions are needed.

Why canonicity matters. Without the right roundtrip, two different byte sequences could decode to the same value. This would break signature verification, deduplication, and protocol-level comparisons. The right roundtrip proof *forced* the discovery of a subtle decoder bug: the nat decoder was accepting multi-byte encodings with the sign bit set (bytes ≥ 192), which the encoder would never produce.

5.2 Halmos: Bytecode Correctness

Halmos symbolically executes the compiled EVM bytecode and proves properties for *all* possible inputs using an SMT solver (Z3). The 40 checks are organized into three suites: **HalmosAsmEquivalence** (30 checks, 1,549s: roundtrip + asm == spec for all types), **HalmosBinaryRoundtrip** (6 checks: address, bytes, keyHash, key, signature, chainId), and **HalmosAdversarial** (4 checks, 610s: adversarial arbitrary bytes for nat/int). The heaviest check is `check_unpackPair_asm_eq_spec` with 2,757 symbolic paths:

Property	Domain	Paths	Time
`toNat(unpack(pack(nat(n)))) == n`	all `uint256`	37	2.5s
`toInt(unpack(pack(int_(v)))) == v`	all `int256`	111	6.6s
`asmNat == specNat`	all `uint256`	37	3.3s
`asmNatRoundtrip`	all `uint256`	37	1.3s
`asmInt == specInt`	all `int256`	74	7.0s
`asmIntRoundtrip`	all `int256`	111	3.0s
`toBool(unpack(pack(bool_(b)))) == b`	all `bool`	3	<0.1s
`asmBool == specBool`	all `bool`	3	<0.1s
`toMutez(unpack(pack(nat(uint256(v))))) == v`	all `uint64`	12	0.5s
`asmMutez == specMutez`	all `uint64`	11	0.4s
`toTimestamp(unpack(pack(int_(int256(v))))) == v`	all `int64`	29	0.8s
`asmTimestamp == specTimestamp`	all `int64`	21	0.8s
`string roundtrip`	symbolic `string`	3	0.8s
`asmPackNat == specPackNat`	all `uint256`	39	2.2s
`asmPackInt == specPackInt`	all `int256`	75	4.5s
Composite types			
`pair roundtrip`	all `(uint256, int256)`	2,757	401s
`asmPair == specPair`	all `(uint256, int256)`	2,757	643s
`or left roundtrip`	all `uint256`	37	1.1s
`or right roundtrip`	all `int256`	74	2.3s
`asmOr left == spec`	all `uint256`	37	2.1s
`asmOr right == spec`	all `int256`	74	4.4s
`option some roundtrip`	all `uint256`	37	1.2s
`option none roundtrip`	—	1	<0.1s
`asmOption some == spec`	all `uint256`	37	2.1s
`asmOption none == spec`	—	1	<0.1s
`list 3-nat roundtrip`	all (`uint256`^{3})	—	—*
`list empty roundtrip`	—	1	<0.1s
`asmList == spec`	all (`uint256`^{2})	—	—*
Binary type roundtrips (HalmosBinaryRoundtrip)			
`address roundtrip`	symbolic `bytes`(22B)	—	—
`bytes roundtrip`	symbolic `bytes`	—	—
`keyHash roundtrip`	symbolic `bytes`(21B)	—	—
`key roundtrip`	symbolic `bytes`	—	—
`signature roundtrip`	symbolic `bytes`	—	—
`chainId roundtrip`	symbolic `bytes4`	—	—
Adversarial (arbitrary malformed bytes)			
`asm accepts iff spec accepts (nat)`	symbolic `bytes`	157	1.7s
`asm rejects iff spec rejects (nat)`	symbolic `bytes`	157	9.8s
`asm accepts iff spec accepts (int)`	symbolic `bytes`	157	1.7s
`asm rejects iff spec rejects (int)`	symbolic `bytes`	157	9.7s

5.3 Combined Trust Chain

There are two unverified manual steps: the transpiler (translating Lean spec to Solidity) and the assembly derivation/inlining. Halmos closes *both* gaps—it proves the reference Solidity satisfies the roundtrip property directly on the bytecode, independently of how it was written, and then proves the assembly produces the same results as the reference. Additionally, 96 differential

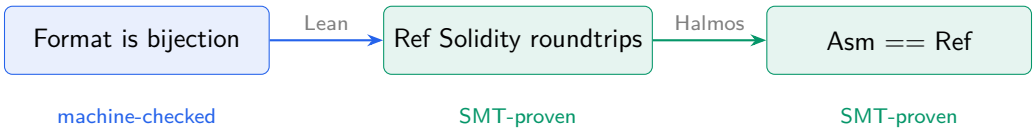

Figure 2: Trust chain. Each link is independently verified.

tests confirm asm == spec across all input ranges.

5.4 Byte Representation: `List (Fin 256)`

The Lean specification models byte sequences as `List (Fin 256)`—each element structurally carries a proof that it is in $[0, 255]$. This eliminates an entire class of lemmas and preconditions:

- The `encode_bytes_valid` family of lemmas (proving encoder outputs are < 256) is **eliminated**: the `Fin 256` return type enforces this by construction.
- Right-roundtrip theorems have **no `hbytes` precondition**: there is no need to assume $\forall b \in \text{bs},\ b < 256$ because the type makes out-of-range values unrepresentable.
- The `Pack.lean` typed API layer is **deleted**: there is no longer a gap between the internal representation and the public interface.
- Invalid inputs like `[999, 0, 0, 0]` are rejected at the type level (not representable as `List (Fin 256)`).

Tactic compatibility. Lean's `omega` tactic works on `Fin.val` (which is a `Nat` with a bound proof) without requiring manual conversion. The migration to `Fin 256` did not add significant proof boilerplate.

6 Optimization Results

The library provides two implementations: a reference (pure Solidity, transpiled from Lean) and an assembly-optimized version (structurally derived from the transpiled spec, with internal functions inlined for performance). The inlined assembly matches the performance of the old monolithic hand-written assembly (within 2%), while maintaining auditable correspondence to the spec via `// - Corresponds to MichelsonSpec.XXX -` comments.

6.1 Gas Comparison

Table 1 shows gas costs for all operations across all types, measured locally via Foundry. Where assembly-optimized implementations exist, both are shown.

Key observations.

- **String decode (256 bytes): 99% savings** (135,881 → 1,048). The reference copies byte-by-byte via `_slice`; the assembly uses word-sized `mload`/`mstore` copies.
- **Nat/int decode (small values): 79–83% savings** (1,546 → 263). The transpiled recursive spec is 10–35% faster than the old iterative spec for decoders because the offset-based pattern avoids `_slice` copy overhead. The inlined assembly eliminates allocation entirely, reading directly from the input buffer.

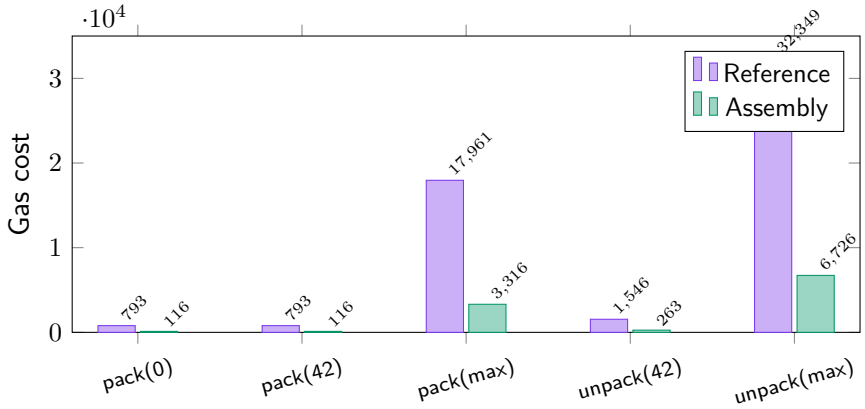

Figure 3: Nat encode/decode gas: reference vs assembly. Savings grow with value size due to the reference's quadratic `abi.encodePacked` allocation.

- **Nat/int decode (large values): 79% savings** (32,349 → 6,726). The offset-based spec decoder is 10% faster than the previous version for large values.
- **Nat/int encode (large values): 82% savings** (17,961 → 3,316). The reference calls `abi.encodePacked` 37 times in a loop ($O(n^2)$ copying); the assembly writes to a single pre-allocated buffer.
- **Nat/int encode (small values): 85%**. Even for single-byte encodings, the assembly saves by avoiding Solidity's `abi.encodePacked` overhead (116 vs 793 gas).
- **Bool decode: 85% savings** (1,432 → 217).
- **Inlined asm matches old monolithic asm.** The inlined assembly (derived from the transpiled spec with correspondence comments) achieves performance within 2% of the old hand-written monolithic assembly. The structural derivation adds no overhead.
- **String encode: no savings.** The assembly `pack(string_())` is slightly slower than the reference for short strings (563 vs 504) because the reference uses `abi.encodePacked` which is optimized for concatenation. For long strings, the assembly's word-sized copy is also slower (995 vs 384) because the reference benefits from the compiler's built-in memory copy. The decode direction is where string savings matter most.
- **Mutez/timestamp: 71–82%**. These delegate to the nat/int assembly decoders with an additional bounds check.
- **Composite types: no assembly optimization.** There are no assembly-optimized versions of the composite encode/decode functions. The reason is architectural: composite decoding requires `_michelineNodeSize`, a recursive function that dispatches on Micheline node tags (Int, String, Prim with 0/1/2 args, Sequence, Bytes) and now includes a `depth` parameter (max 64 levels) to prevent stack overflow DoS from deeply nested Micheline nodes. Yul does not support recursive function calls, so implementing this would require a manual call stack—adding complexity without meaningful gas savings, since the composite wrappers are already cheap (600–5,200 gas).

  In practice, the gas-critical path is the *inner value* decoding. A typical usage pattern is:

  1. `toPair(unpack(packed))` → split into two Micheline nodes (reference Solidity, 5,228 gas)
  2. `Michelson.unpackNat(left)` → decode inner nat (assembly, 263 gas instead of 1,546)

3. `Michelson.unpackString(right)` → decode inner string (assembly, 446 gas instead of 2,780)

The composite wrapper adds fixed overhead; the atom-level assembly saves 79–99% on each inner decode where the actual computation happens.

7 Testing

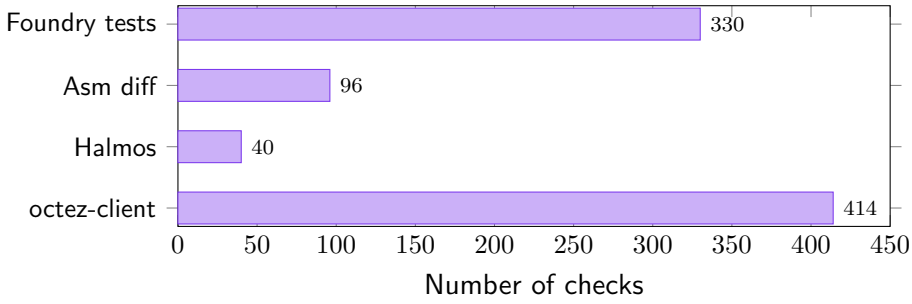

Figure 4: Test coverage by layer (∼880 total checks).

Category	Tool	Checks
Foundry tests (all types)	Foundry fuzz	330 pass / 330 total
including: Asm == Reference	Foundry fuzz	96
Symbolic proofs (all inputs)	Halmos	40
Tezos node compatibility	octez-client	414
Total		**∼880**

8 Usage

8.1 Basic Encoding/Decoding

```solidity
import {Michelson} from "./Michelson.sol";

contract Example {
    function callMichelson(uint256 amount) external {
        bytes memory packed = Michelson.pack(Michelson.nat(amount));
        // Send `packed` to Michelson runtime...
    }

    function handleCallback(bytes memory packed) external {
        uint256 value = Michelson.toNat(Michelson.unpack(packed));
    }
}
```

Note on assembly optimization. The production `Michelson.sol` library uses assembly-optimized internals for atom types (79–99% gas savings). The assembly is structurally derived from `MichelsonSpec.sol` with inlined functions and spec correspondence comments. Users import a single library (`Michelson`) and get the optimized path automatically. The reference implementation (`MichelsonSpec.sol`), transpiled from the Lean specification, exists for Halmos verification and testing.

8.2 Composing Composite Types

The library uses a two-level API for composite types. Inner `encode*` functions produce raw Micheline bytes (no `0x05` prefix), which are composable. Outer `pack*` functions add the version byte at the top level:

Composing Pair(nat

```
// Inner pair: Pair("hello", true)  no 0x05 prefix
bytes memory inner = Michelson.pair(
    Michelson.string_("hello"),
    Michelson.bool_(true)
);
// Outer pack: 0x05 + Pair(42, inner)
bytes memory packed = Michelson.pack(Michelson.pair(
    Michelson.nat(42),
    inner
));
// Unpack: split into two Micheline nodes
(bytes memory left, bytes memory right) =
    Michelson.toPair(Michelson.unpack(packed));
// left = nat(42) = 0x00 0x2a
// right = pair(string_("hello"), bool_(true))
```

The same pattern works for `or` (Left/Right routing), `option` (Some/None), and `list` (variable-length sequences):

Or

```
// Or: Left(42) for routing
bytes memory orVal = Michelson.pack(Michelson.left(
    Michelson.nat(42)));

// Option: Some("hello")
bytes memory opt = Michelson.pack(Michelson.some(
    Michelson.string_("hello")));

// List: [1, 2, 3]
bytes[] memory items = new bytes[](3);
items[0] = Michelson.nat(1);
items[1] = Michelson.nat(2);
items[2] = Michelson.nat(3);
bytes memory list = Michelson.pack(Michelson.list(items));
```

9 Conclusion

The Michelson PACK codec for Tezos X provides verified cross-runtime serialization with an honest end-to-end pipeline:

1. The encoding format is a proven bijection (Lean 4, machine-checked).
2. The Solidity reference is transpiled from the proven Lean specification.
3. The assembly is structurally derived from the transpiled spec, inlined for performance, and annotated with spec correspondence markers.
4. The Solidity bytecode satisfies the roundtrip property for all inputs (Halmos, SMT-proven).

5. The assembly is proven equivalent to the reference (Halmos, SMT-proven; 96 differential tests confirm asm == spec).

6. The output is compatible with the Tezos reference node (`octez-client`, 414 differential tests across all 20 types).

Every step is connected: Lean spec → proven equivalent → Implementation → transpiled → Solidity spec → derived → assembly. There are no manual translation gaps. Halmos verifies spec correctness, and differential tests verify asm == spec.

The two-layer approach—Lean for the format, Halmos for the bytecode—provides complementary guarantees. Lean catches design-level issues (like the nat sign-bit canonicity bug described in Appendix **??**) that no implementation testing would find. Halmos catches implementation-level issues in the actual deployed artifact. Together they close the verification chain from mathematical specification to on-chain bytecode.

10 Tutorial: Using the Library

This section walks through the most common usage patterns for the Michelson PACK codec. All examples use the reference library (`Michelson`). For gas-critical paths, replace atom-level calls with `Michelson`.

10.1 Setup

Import

```
import {Michelson} from "./Michelson.sol";
```

10.2 Encoding and Decoding Atom Values

Atom encoding functions (e.g., `Michelson.nat(42)`) produce raw Micheline bytes. Wrap with `Michelson.pack(...)` for top-level PACK format, or nest inside composites directly. Decode with `Michelson.toNat(...)`, `Michelson.toString(...)`, etc.

Atom types

```
// Nat
bytes memory packed = Michelson.pack(Michelson.nat(42));
uint256 n = Michelson.toNat(Michelson.unpack(packed)); // 42

// Int
bytes memory p = Michelson.pack(Michelson.int_(-100));
int256 v = Michelson.toInt(Michelson.unpack(p)); // -100

// Bool
bytes memory b = Michelson.pack(Michelson.bool_(true));
bool flag = Michelson.toBool(Michelson.unpack(b)); // true

// String
bytes memory s = Michelson.pack(Michelson.string_("hello"));
string memory str = Michelson.toString(Michelson.unpack(s));

// Unit
bytes memory u = Michelson.pack(Michelson.unit_());
Michelson.toUnit(Michelson.unpack(u)); // reverts if invalid

// Mutez (bounded to 0 .. 2^63-1)
bytes memory m = Michelson.pack(Michelson.nat(uint256(1_000_000)));
uint64 amount = Michelson.toMutez(Michelson.unpack(m));

// Timestamp (bounded to int64 range)
bytes memory t = Michelson.pack(Michelson.int_(int256(-3600)));
int64 ts = Michelson.toTimestamp(Michelson.unpack(t));

// Bytes
bytes memory raw = hex"deadbeef";
bytes memory bp = Michelson.pack(Michelson.bytes_(raw));
bytes memory decoded = Michelson.toBytes(Michelson.unpack(bp));
```

10.3 Binary Types (address, key, signature, etc.)

Binary types take **raw bytes**, not base58check strings. Base58check encoding is too expensive in Solidity; the caller handles conversion off-chain.

Binary types

```
// Address: 22-byte binary (NOT "tz1..." string)
bytes memory addr = hex"00006b82198cb179e8306c1bedd08f12dc863f328886";
bytes memory ap = Michelson.pack(Michelson.address_(addr));

// Key hash: 21 bytes (1-byte tag + 20-byte hash)
bytes memory kh = hex"006b82198cb179e8306c1bedd08f12dc863f328886";
bytes memory khp = Michelson.pack(Michelson.keyHash(kh));

// Chain ID: 4 bytes
bytes memory cid = Michelson.pack(Michelson.chainId(bytes4(hex"7a06a770")));

// Contract: address + optional entrypoint
bytes memory cp = Michelson.pack(Michelson.contract_(addr, "transfer"));
```

10.4 Map and Set

Map: key-value pairs

```
// Map { Elt 1 "a", Elt 2 "b" }
bytes[] memory elts = new bytes[](2);
elts[0] = Michelson.elt(
    Michelson.nat(1),
    Michelson.string_("a"));
elts[1] = Michelson.elt(
    Michelson.nat(2),
    Michelson.string_("b"));
bytes memory packed = Michelson.pack(Michelson.map(elts));

// Set: identical to list at binary level
bytes[] memory items = new bytes[](3);
items[0] = Michelson.nat(1);
items[1] = Michelson.nat(2);
items[2] = Michelson.nat(3);
bytes memory setP = Michelson.pack(Michelson.set(items));
```

10.5 The Two-Level API: `encode*` vs `pack*`

When building composite types (pair, or, option, list), you must use `encode*` for inner values and `pack*` only at the outermost level. The difference:

Function	Output	When to use
`nat(42)`	`0x00 0x2a`	Inside a pair, or, option, or list
`pack(nat(42))`	`0x05 0x00 0x2a`	Top-level (standalone value)

Rule: `pack*` = `0x05` + `encode*`. Never nest a `pack*` inside another composite — it would produce a double `0x05` prefix.

10.6 Building Composite Types

Pair: two-argument entrypoint

```
// Pack Pair(42, "hello") for a Michelson entrypoint
// expecting (pair nat string)
bytes memory packed = Michelson.pack(Michelson.pair(
    Michelson.nat(42),
    Michelson.string_("hello")
));

// Unpack: get the two inner Micheline nodes
(bytes memory left, bytes memory right) =
    Michelson.toPair(Michelson.unpack(packed));

// Decode each inner value using toNat/toString
// (add 0x05 prefix to convert from Micheline to PACK)
uint256 n = Michelson.toNat(Michelson.unpack(
    abi.encodePacked(hex"05", left)));
string memory s = Michelson.toString(Michelson.unpack(
    abi.encodePacked(hex"05", right)));
```

Or: routing Left/Right

```
// Send Left(42) to a (or nat string) entrypoint
bytes memory packed = Michelson.pack(Michelson.left(
    Michelson.nat(42)));

// Receive and dispatch
(bool isLeft, bytes memory inner) =
    Michelson.toOr(Michelson.unpack(packed));
if (isLeft) {
    uint256 val = Michelson.toNat(Michelson.unpack(
        abi.encodePacked(hex"05", inner)));
    // handle nat...
} else {
    string memory val = Michelson.toString(Michelson.unpack(
        abi.encodePacked(hex"05", inner)));
    // handle string...
}
```

Option: nullable values

```
// Some(42)
bytes memory some = Michelson.pack(Michelson.some(
    Michelson.nat(42)));

// None
bytes memory none = Michelson.pack(Michelson.none());

// Decode
(bool isSome, bytes memory inner) =
    Michelson.toOption(Michelson.unpack(someOrNone));
if (isSome) {
    uint256 val = Michelson.toNat(Michelson.unpack(
        abi.encodePacked(hex"05", inner)));
}
```

List: variable-length sequences

```
// Encode [1, 2, 3] as list nat
bytes[] memory items = new bytes[](3);
items[0] = Michelson.nat(1);
items[1] = Michelson.nat(2);
items[2] = Michelson.nat(3);
bytes memory packed = Michelson.pack(Michelson.list(items));

// Decode: get the raw payload (concatenated elements)
bytes memory payload =
    Michelson.toList(Michelson.unpack(packed));
// Parse individual elements from payload using
// _michelineNodeSize or application-level knowledge
// of the element type and count.
```

10.7 Nesting Composite Types

Composite types nest naturally. Inner composites use `encode*` (no `0x05` prefix), and only the outermost call uses `pack*`:

Nested: Pair(nat

```
bytes memory inner = Michelson.pair(
    Michelson.string_("hello"),
    Michelson.bool_(true)
);
bytes memory packed = Michelson.pack(Michelson.pair(
    Michelson.nat(42),
    inner
));
// Result: 0x05 0707 002a 0707 01000000568656c6c6f 030a
```

10.8 Decoding Composite Values

All functions are in the single `Michelson` library. Atom-level operations use assembly internally (79–99% gas savings). Composite operations use pure Solidity (recursive Micheline parsing).

Decoding Pair(nat

```
// Unpack pair into two Micheline nodes
(bytes memory left, bytes memory right) =
    Michelson.toPair(Michelson.unpack(packed));

// Decode each inner value (asm-optimized internally)
uint256 n = Michelson.toNat(left);
string memory s = Michelson.toString(right);
```

10.9 Error Handling

All decode functions revert with typed errors on invalid input:

Catching decode errors

```
try Michelson.toNat(Michelson.unpack(untrustedBytes))
    returns (uint256 value) {
    // success  use value
} catch (bytes memory err) {
    // decode failed  invalid PACK bytes
    // err contains the error selector:
    // InputTruncated, InvalidVersionByte,
    // UnexpectedNodeTag, IntOverflow,
    // NatNegative, TrailingBytes, etc.
}
```

Table 1: Gas cost per operation. Assembly variants exist for all 7 types. Savings range from 47% (unit pack) to 99% (string decode 256 B). The inlined assembly matches the old monolithic assembly performance (within 2%).

Type	**Operation**	**Ref (gas)**	**Asm (gas)**	**Savings**
5*nat	`pack(nat(0))`	793	116	85%
	`pack(nat(42))`	793	116	85%
	`pack(nat(uint256.max))`	17,961	3,316	82%
	`toNat(unpack(42))`	1,546	263	83%
	`toNat(unpack(uint256.max))`	32,349	6,726	79%
4*int	`pack(int_(-1))`	793	116	85%
	`pack(int_(int256.min))`	17,950	3,310	82%
	`toInt(unpack(-42))`	1,525	324	79%
	`toInt(unpack(int256.min))`	32,330	6,781	79%
2*bool	`pack(bool_(true))`	173	80	54%
	`toBool(unpack(true))`	1,432	217	85%
2*unit	`pack(unit_())`	150	80	47%
	`toUnit(unpack())`	499	155	69%
4*string	`pack(string_("hello"))`	504	563	—
	`toString(unpack("hello"))`	2,780	446	84%
	`pack(string_(256 B))`	384	995	—
	`toString(unpack(256 B))`	135,881	1,048	**99%**
2*mutez	`pack(nat(uint256(1M)))`	1,439	274	81%
	`toMutez(unpack(max))`	7,568	1,948	74%
2*timestamp	`pack(int_(int256(-1M)))`	1,545	274	82%
	`toTimestamp(unpack(-1M))`	2,831	823	71%
2*packToEVM	`packToEVMNat(max)`	25,883	6,615	74%
	`packToEVMInt(min)`	25,895	6,698	74%
Composite types (no assembly optimization — see below)				
pair	`pack(pair(nat,string))`	600	—	
	`toPair(unpack)`	5,228	—	
or	`pack(left(nat))`	517	—	
	`toOr(unpack(left))`	1,518	—	
option	`pack(some(nat))`	517	—	
	`toOption(unpack(some))`	1,502	—	
list	`pack(list({1,2,3}))`	1,940	—	
	`toList(unpack({1,2,3}))`	3,040	—	